



Carrera: Ingeniería de Software con IA

FUNDAMENTOS Y ALGORITMIA

PARA INTELIGENCIA ARTIFICIAL



RESULTADO DE APRENDIZAJE:

Al finalizar la sesión, el estudiante explica el fundamento probabilístico de Naive Bayes, calcula probabilidades posteriores mediante el Teorema de Bayes y comprende cómo el modelo utiliza la independencia condicional para realizar clasificaciones.

CONTENIDO:

- Fundamentos de probabilidad
- Probabilidad condicional
- Regla del producto Teorema de Bayes
- Aplicaciones de Bayes en IA
- Clasificador Naive Bayes

✉ **Caso de Estudio**

Un correo contiene

“GANADOR – GRATIS – OFERTA”

¿La IA puede afirmar categóricamente que es spam?



Probabilidad condicionada

Supuestos del Problema

- ✓ 30% de correos son spam.
- ✓ 80% de los spam contienen "gratis".
- ✓ 20% de los correos legítimos contienen "gratis".

Queremos conocer:

$P(\text{Spam} \mid \text{Gratis})$

Pero conocemos fácilmente:

$P(\text{Gratis} \mid \text{Spam})$

¿Son iguales $P(\text{Spam} \mid \text{Gratis})$ y $P(\text{Gratis} \mid \text{Spam})$?

Probabilidad condicional

$$P(A \mid B)$$

Probabilidad de que ocurra A sabiendo que B ya ocurrió.

$P(\text{Spam} \mid \text{Gratis})$

$P(\text{Enfermo} \mid \text{Positivo})$

$P(\text{Lluvia} \mid \text{Nubes})$

Probabilidad conjunta y regla del producto

$$P(A \cap B)$$

Probabilidad de que ocurran A y B simultáneamente.

$$P(A \cap B) = P(A | B) \cdot P(B)$$

$$P(A \cap B) = P(B | A) \cdot P(A)$$

- Las dos expresiones representan la misma probabilidad conjunta.
- Esta igualdad será la base algebraica del Teorema de Bayes.

Independencia estadística

Definición

$$P(A \cap B) = P(A) \cdot P(B)$$

$$P(A | B) = P(A)$$

Interpretación: conocer B no modifica la probabilidad de A.

Ejemplo

Lanzar un dado y obtener cara en una moneda.

El resultado del dado no cambia el resultado de la moneda.

¿Las palabras de una oración son independientes?

¿De dónde sale el Teorema de Bayes?

Partimos de:

$$P(A \cap B) = P(A|B)P(B)$$

También:

$$P(A \cap B) = P(B|A)P(A)$$

Igualamos y despejamos:

$$P(A|B) = [P(B|A)P(A)] / P(B)$$

- Probabilidad condicional: $P(A|B)$
- Probabilidad conjunta: $P(A \cap B)$
- Independencia: $P(A \cap B) = P(A) \cdot P(B)$

El Teorema de Bayes

$$P(A | B) = [P(B | A) \cdot P(A)] / P(B)$$

1. PRIOR **Creencia inicial**



2. VEROSIMILITUD **Evidencia
observada**



3. POSTERIOR **Creencia actualizada**

Componentes:

- $P(A)$: probabilidad previa (prior)
- $P(B|A)$: verosimilitud (likelihood)
- $P(A|B)$: probabilidad posterior
- $P(B)$: evidencia

Los componentes del Teorema

Componente	Nombre	Pregunta
$P(A)$	Prior	¿Qué tan frecuente era A antes de B?
$P(B A)$	Likelihood	¿Qué tan probable es B si A es cierto?
$P(B)$	Evidence	¿Qué tan frecuente es B en general?
$P(A B)$	Posterior	¿Qué tan probable es A después de B?

Antes de observar la evidencia, tenemos una creencia base (**Prior**).

Después de medir la evidencia y ponderarla, ajustamos nuestra perspectiva obteniendo el **Posterior**.

Los componentes del Teorema

Ejemplo médico

- 1% de la población tiene una enfermedad.
- El test detecta al 99% de los enfermos.
- El 5% de los sanos da positivo por error.

Si el test es positivo, ¿cuál es la probabilidad de estar enfermo?

$$P(\text{Enfermo} \mid +) = ?$$

Primero: estima el resultado antes de calcularlo.

Los componentes del Teorema

Ejemplo médico: **10 000 personas**



$$P(\text{Enfermo} \mid +) = 99 / (99 + 495) \approx 16.7\%$$

La intuición puede fallar porque el prior es pequeño y los falsos positivos son numerosos.

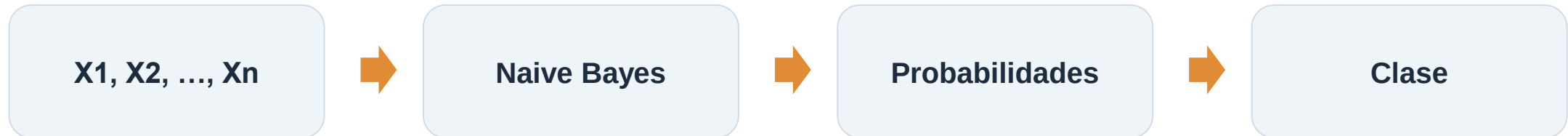
¿Qué es Naive Bayes?

Algoritmo de clasificación supervisado basado en el Teorema de Bayes.

Recibe características $X_1, X_2, \dots, X_n$.

Calcula qué tan probable es cada clase.

La predicción final corresponde a la clase con mayor probabilidad posterior.



¿Por qué “Naive”?

Se llama **Naive Bayes** porque “naive” significa “ingenuo”.

La idea “ingenua” del modelo es asumir que **las características de un dato son independientes entre sí**, una vez conocida la clase.

Supuesto Matemático:

$$P(X_1, X_2, \dots, X_n \mid C) \approx \prod_i P(X_i \mid C)$$

Las características se consideran condicionalmente independientes dada la clase.

¿Por qué “Naive”?

Por ejemplo, para clasificar un correo como **spam**:

- Tiene la palabra “gratis”.
- Tiene la palabra “premio”.
- Tiene la palabra “ganador”.

Naive Bayes supone que estas características pueden tratarse **como independientes** al calcular la probabilidad de que el correo sea spam.

¿Por qué “Naive”?

Bayes = utiliza el Teorema de Bayes.

Naive = hace el supuesto simplificador de **independencia condicional entre las características**

Aunque este supuesto rara vez se cumple perfectamente en la realidad, el modelo puede funcionar bastante bien en problemas de clasificación.

Fórmula general de Naive Bayes

$$P(C | X_1, \dots, X_n) \propto P(C) \cdot \prod_i P(X_i | C)$$

Prior **$P(C)$**

×

Evidencia 1 **$P(X_1 | C)$**

×

Evidencia 2... **$P(X_2 | C)$**

Prior **$P(C)$**



Evidencia 1



Evidencia 2



Evidencia 3

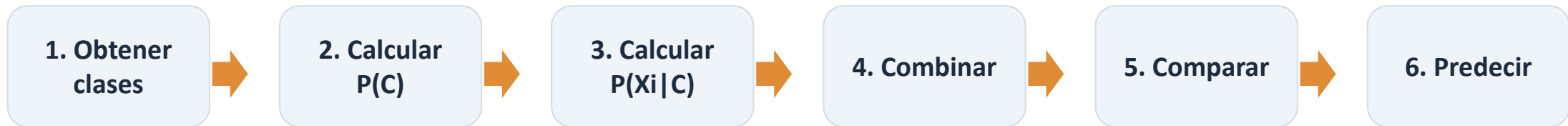


Puntuación

Se repite el cálculo para cada clase disponible.

La clase con mayor puntuación es la predicción.

Proceso de Clasificación



$$\hat{C} = \operatorname{argmax}_C P(C | X)$$

Ejemplo

Clima	Temperatura	Humedad	Jugar
Soleado	Alta	Alta	No
Nublado	Alta	Alta	Sí
Lluvia	Media	Alta	Sí
Lluvia	Baja	Normal	Sí
Soleado	Media	Normal	Sí
...	...	...	...

**Nuevo caso:
Soleado – Media – Normal**

Ejemplo

Clima	Temperatura	Humedad	Jugar
Soleado	Alta	Alta	No
Nublado	Alta	Alta	Sí
Lluvia	Media	Alta	Sí
Lluvia	Baja	Normal	Sí
Soleado	Media	Normal	Sí
...	...	...	...

Para la clase **Sí**:

$$P(\text{Sí}) \times P(\text{Soleado} \mid \text{Sí}) \times P(\text{Media} \mid \text{Sí}) \times P(\text{Normal} \mid \text{Sí})$$

Para la clase **No**:

$$P(\text{No}) \times P(\text{Soleado} \mid \text{No}) \times P(\text{Media} \mid \text{No}) \times P(\text{Normal} \mid \text{No})$$

Problema computacional: Underflow

Con muchas características, las probabilidades pueden ser extremadamente pequeñas.

Ejemplo:

$$0.01 \times 0.003 \times 0.002 \times \dots$$

El resultado puede quedar tan cerca de cero que la computadora pierde precisión.

Esto se conoce como subdesbordamiento numérico o underflow.

**Muchas multiplicaciones pequeñas
→ pérdida de precisión**

Logaritmos y estabilidad numérica

De producto a suma:

$$\log P(C) + \sum_i \log P(X_i | C)$$

Basado en la propiedad: $\log(a \cdot b) = \log(a) + \log(b)$

Multiplicaciones



logaritmos



Sumas



Mayor estabilidad

Elimina por completo el underflow numérico y dota al modelo de una estabilidad matemática robusta en producción.

Variantes de Naive Bayes

Adaptando el algoritmo según la naturaleza de los datos.

GaussianNB

Datos continuos con distribución normal.

MultinomialNB

Conteos discretos y frecuencia de palabras.

BernoulliNB

Características binarias (presente/ausente).

ComplementNB

Especial para texto con clases desbalanceadas.

Gaussian Naive Bayes

- Se utiliza con características continuas.
- Para cada característica y clase estima parámetros de una distribución normal.
- Los parámetros principales son la media μ y la varianza σ^2 .

Ejemplo:

Edad, Ingresos, Temperatura, Altura, Peso.

Clase A
 $\mu = 25$ $\sigma = 3$

Clase B
 $\mu = 42$ $\sigma = 6$

Evaluación en la Curva Normal

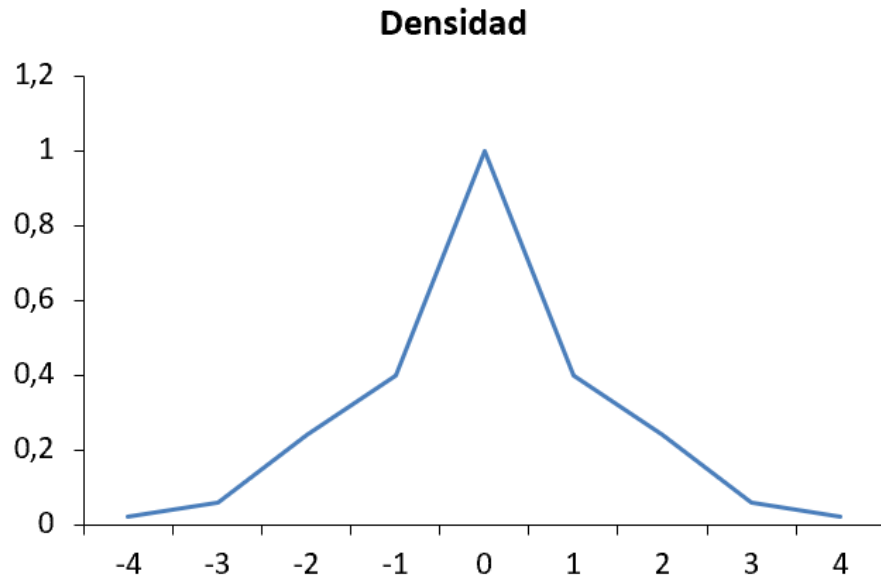
¿Cómo calcula la verosimilitud un valor continuo?

La curva normal permite modelar cómo se distribuye una característica dentro de una clase.

Por ejemplo, si queremos clasificar alumnos como “**aprobado**” o “**desaprobado**”, podemos analizar cómo se distribuyen sus notas en cada clase.

En Naive Bayes, cuando se trabaja con variables continuas, el **Gaussian Naive Bayes** puede asumir que cada característica sigue una **distribución normal dentro de cada clase**.

Evaluación en la Curva Normal



Densidad

Evalúa la altura de la función de densidad de probabilidad en el punto exacto aportado por la característica del nuevo registro.

Probabilidad cero

Supongamos que “Bitcoin” nunca apareció en los documentos de la clase No Spam.

Entonces $P(\text{Bitcoin} \mid \text{No Spam}) = 0$.

Como Naive Bayes multiplica probabilidades, la puntuación de esa clase puede quedar en cero.

$$P(\text{Bitcoin} \mid \text{NoSpam}) = 0$$

Como se multiplican probabilidad: toda la probabilidad posterior se anula a cero.

Suavizado de Laplace (Add-1)

$$P(w \mid C) = [\text{count}(w,C) + \alpha] / [\text{count}(C) + \alpha V]$$

Evita probabilidades exactamente iguales a cero.

α es un hiperparámetro.

$\alpha = 1$ es un caso común; otros valores pueden ajustarse.

V representa el tamaño del vocabulario o número de categorías posibles en la representación.

Modelos Generativos vs. Discriminativos

Generativo (Naive Bayes)

Modela $P(X | C)$.

Describe cómo se generan los datos para cada clase en particular.

Discriminativo (Regresión Logística)

Modela $P(C | X)$.

Se enfoca exclusivamente en encontrar la frontera de decisión entre clases.

El problema real de la independencia

Ejemplo lingüístico: “Nueva York”.

La aparición de “Nueva” cambia la probabilidad de encontrar “York”.

En datos tabulares también pueden existir relaciones entre edad, ingresos, educación, experiencia, etc.

Por tanto, la independencia condicional es una simplificación del mundo real.

**Supuesto del modelo $\neq$
realidad**

¿Por qué funciona a pesar del error?

- El objetivo de clasificación no siempre exige modelar perfectamente la realidad.
- Las características individuales pueden seguir aportando evidencia útil.
- Lo importante puede ser conservar el orden relativo de las puntuaciones entre clases.
- Su simplicidad reduce la cantidad de parámetros que deben estimarse.
- En alta dimensionalidad, esa simplicidad puede ser una ventaja.

Supuestos simplificados → clasificación útil

Condición suficiente

Basta con que se cumpla:

$$\text{Score(Clase A)} > \text{Score(Clase B)}$$

Naive Bayes en scikit-learn

```
from sklearn.naive_bayes import GaussianNB

modelo = GaussianNB()

modelo.fit(X_train, y_train)

y_pred = modelo.predict(X_test)
```

.fit() → aprende parámetros a partir de los datos.

.predict() → asigna una clase a cada observación.

La teoría se convierte en un objeto de modelo que podemos entrenar y evaluar.

Extracción de Probabilidades

```
y_pred_prob = modelo.predict_proba(X_test)

print(y_pred_prob[:3])
```

```
# Ejemplo
[[0.82, 0.18],
 [0.15, 0.85],
 [0.91, 0.09]]
```

Fila 1 → 82% clase 0, 18% clase 1.

Fila 2 → 15% clase 0, 85% clase 1.

Fila 3 → 91% clase 0, 9% clase 1.

Estas probabilidades representan la idea de $P(C | X)$ que estudiamos en Bayes.

MultinomialNB para texto

```
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.naive_bayes import MultinomialNB
```

```
vectorizer = TfidfVectorizer()
```

```
X_train_vec = vectorizer.fit_transform(X_train)  
X_test_vec = vectorizer.transform(X_test)
```

```
modelo = MultinomialNB()  
modelo.fit(X_train_vec, y_train)
```

```
y_pred = modelo.predict(X_test_vec)
```

El Hiperparámetro Alpha en Scikit-Learn

```
from sklearn.naive_bayes import MultinomialNB
```

```
modelo_1 = MultinomialNB(alpha=1.0)
```

```
modelo_2 = MultinomialNB(alpha=0.1)
```

```
modelo_1.fit(X_train_vec, y_train)
```

```
modelo_2.fit(X_train_vec, y_train)
```

- alpha controla el suavizado.
- Cambiar alpha cambia el comportamiento del modelo.
- Puede tratarse como hiperparámetro y ajustarse mediante validación.

Matriz de confusión

Desglosando aciertos y errores del clasificador.

	Pred. 0	Pred. 1
Real 0	TN	FP
Real 1	FN	TP

```
from sklearn.metrics import confusion_matrix  
  
cm = confusion_matrix(y_test, y_pred)
```

Métricas de rendimiento

Precisión, Recall y F1-Score en clasificación.

```
from sklearn.metrics import  
    classification_report  
  
print(classification_report(y_test, y_pred))
```

- Precision → de las predicciones positivas, ¿cuántas fueron correctas?
- Recall → de los positivos reales, ¿cuántos detectamos?
- F1-score → equilibrio entre precision y recall.
- Support → cantidad de observaciones de cada clase.

¿Por qué **no basta con el Accuracy** en problemas reales? Porque falla estrepitosamente ante clases desbalanceadas.

Evaluación con ROC-AUC

- ROC-AUC evalúa la capacidad de discriminación utilizando las puntuaciones probabilísticas.
- Un valor mayor indica mejor separación de clases en el conjunto evaluado.
- La interpretación debe considerar el problema y la distribución de clases.

```
y_prob = modelo.predict_proba(X_test)[:, 1]

from sklearn.metrics import roc_auc_score

roc_auc = roc_auc_score(y_test, y_prob)
```

Naive Bayes vs. KNN vs. Regresión Logística

Aspecto	Naive Bayes	KNN	Reg. Logística
Idea	Probabilidad	Distancia	Frontera
Entrenamiento	Rápido	Mínimo	Moderado
Predicción	Rápida	Más costosa	Rápida
Escalamiento	Generalmente no crítico	Muy importante	Recomendado
Texto	Muy bueno	Poco adecuado	Muy bueno
Supuesto	Independencia condicional	Vecindad	Modelo logístico

¿Cuándo utilizar Naive Bayes?

- Muchas características.
- Pocos datos de entrenamiento.
- Necesidad de entrenamiento rápido.
- Clasificación de texto.
- Construcción de un baseline para comparar modelos.
- Necesidad de un modelo sencillo y computacionalmente eficiente.

No se elige solo porque sea rápido.

Datos + problema + objetivo + métricas

Casos de uso en la Industria



Filtros Anti-Spam

Correo → TF-IDF → Multinomial NB



Categorización de Noticias

Deportes, Economía, Política



Análisis de Sentimiento

Opiniones de usuarios y reseñas

Pregunta: ¿qué modelo elegirías?

Métrica	Modelo A	Modelo B
Accuracy	94%	92%
F1	0.82	0.89
ROC-AUC	0.91	0.94

¿Qué métrica importa más?

¿Existe desbalance?

¿Cuál es el costo de FP y FN?

¿Necesitamos probabilidades calibradas?

¿Cuál es el objetivo del sistema?

**“La cercanía también
es información”**